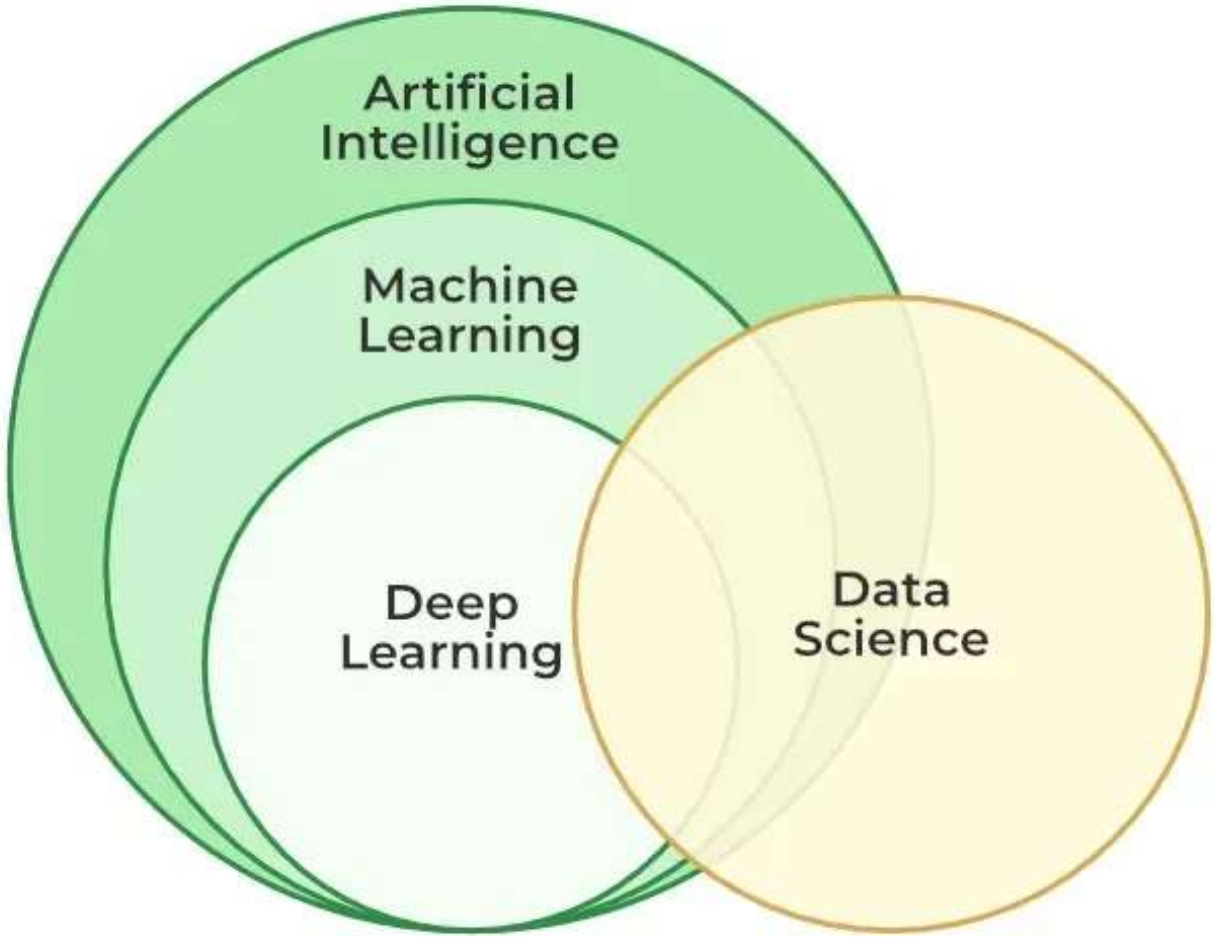
DEEP LEARNING USING PYTHON


Artificial Intelligence
Machine Learning
Deep Learning
Data Science

WHEN TO CHOOSE DEEP LEARNING

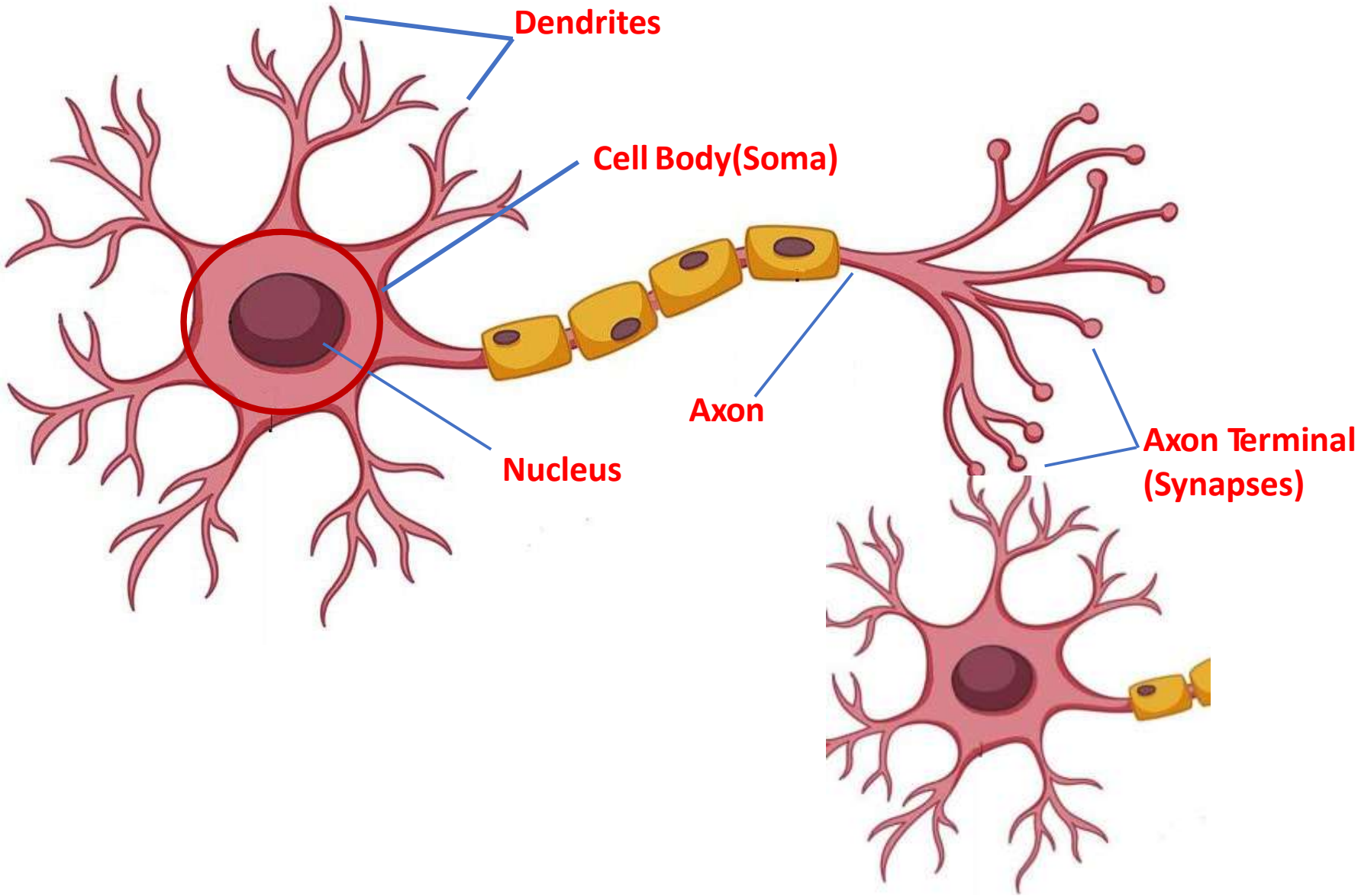
Classic Programming

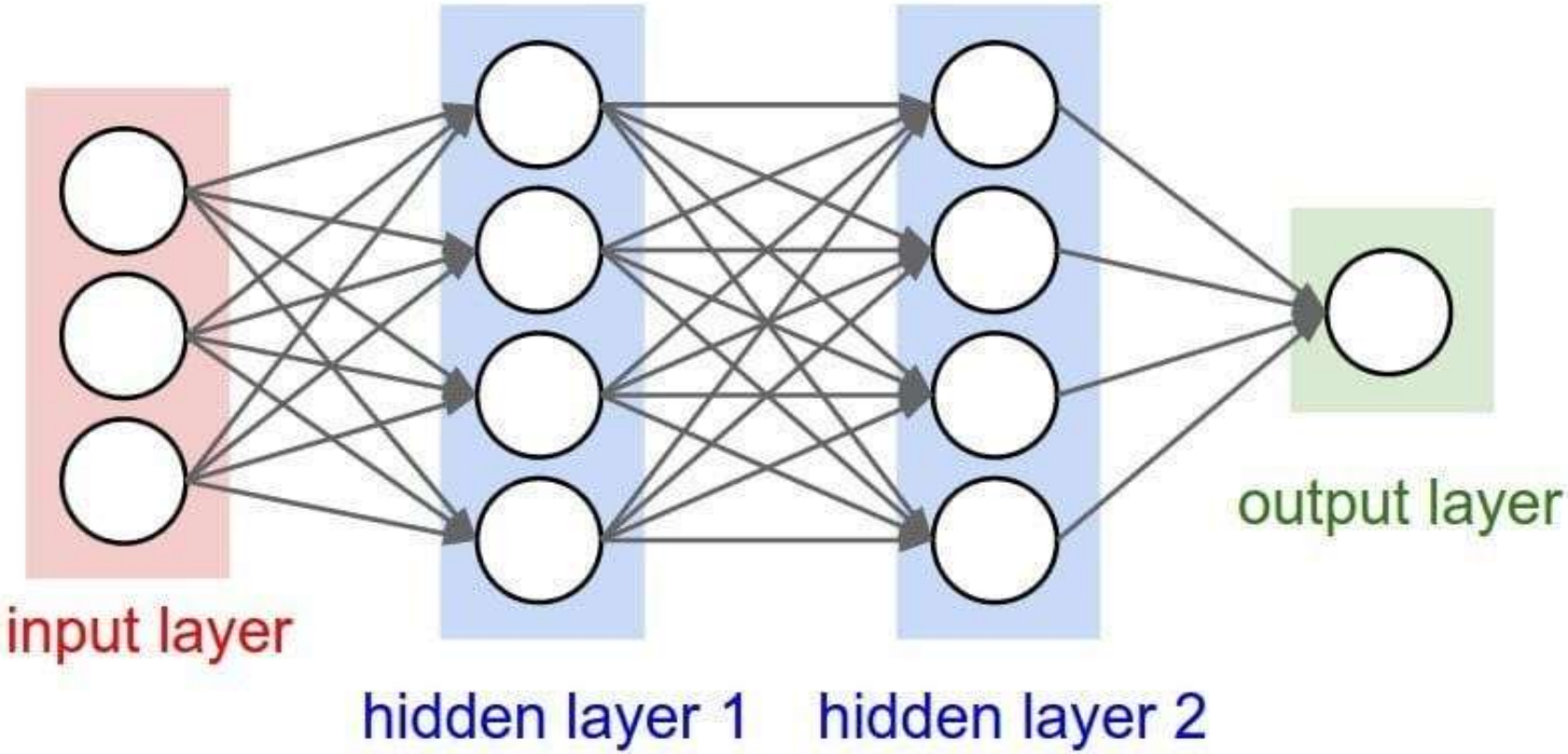
If rules are clear and straightforward, often better to just program it

Deep Learning

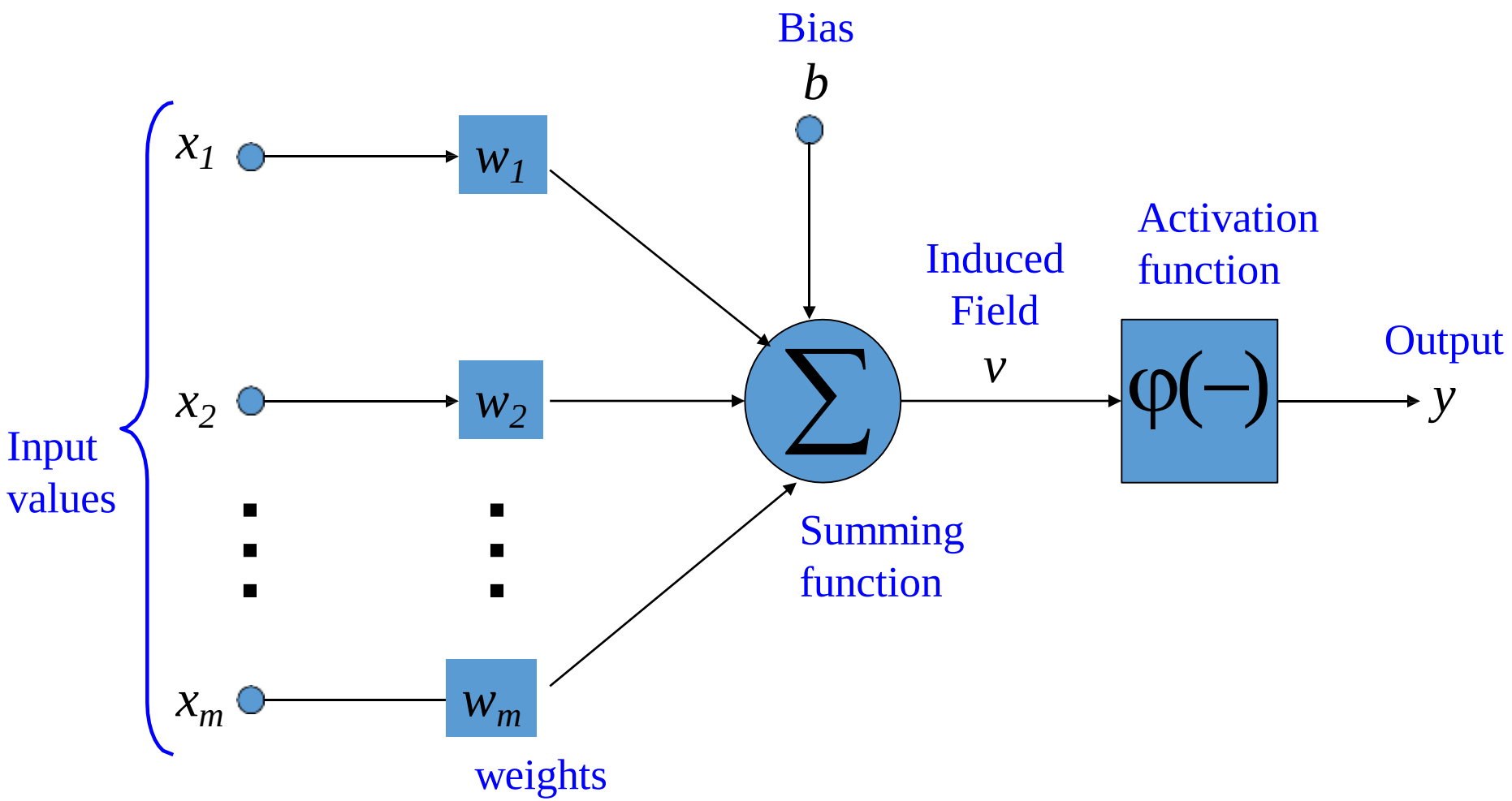
If rules are complex and difficult, use deep learning

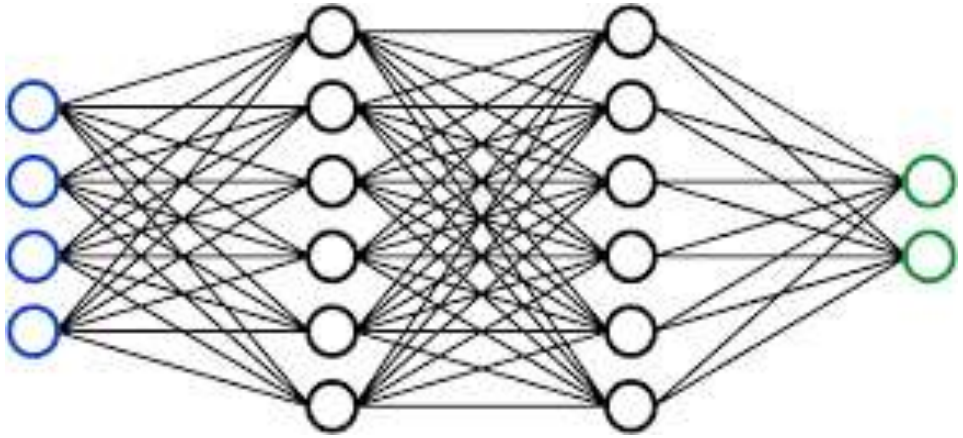
Deep Learning


Dendrites
Cell Body(Soma)
Axon
Nucleus
Axon Terminal
(Synapses)


input layer
hidden layer 1
hidden layer 2
output layer

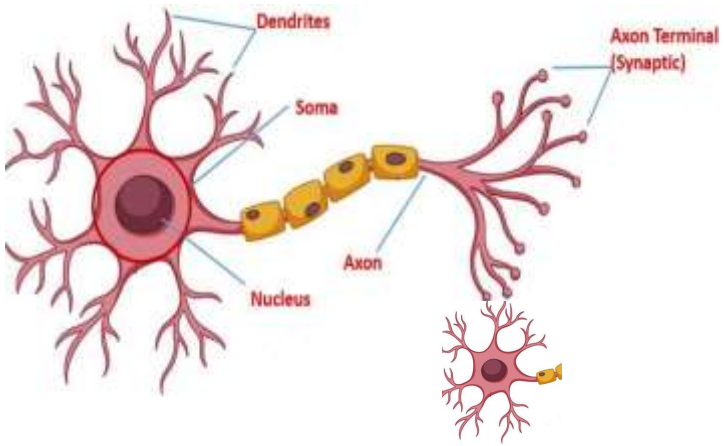
Perceptron

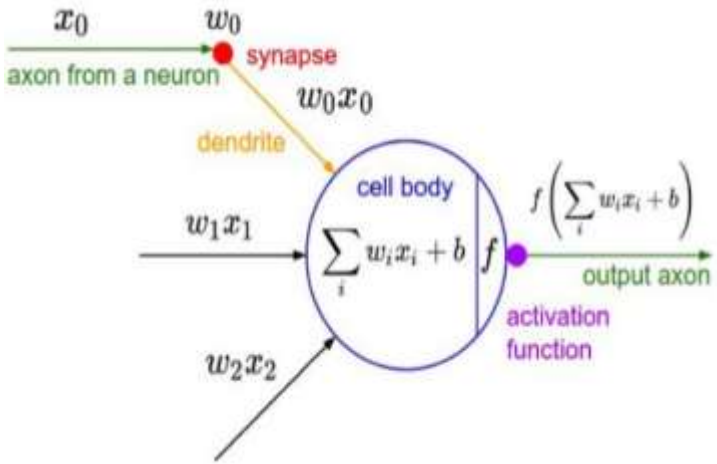

Bias
b
x_1
x_2
x_m
w_1
w_2
w_m
Input values
weights
Σ
Summing function
Induced Field
v
Activation function
$\varphi(-)$
Output
y



Network of neurons

Network of neurons



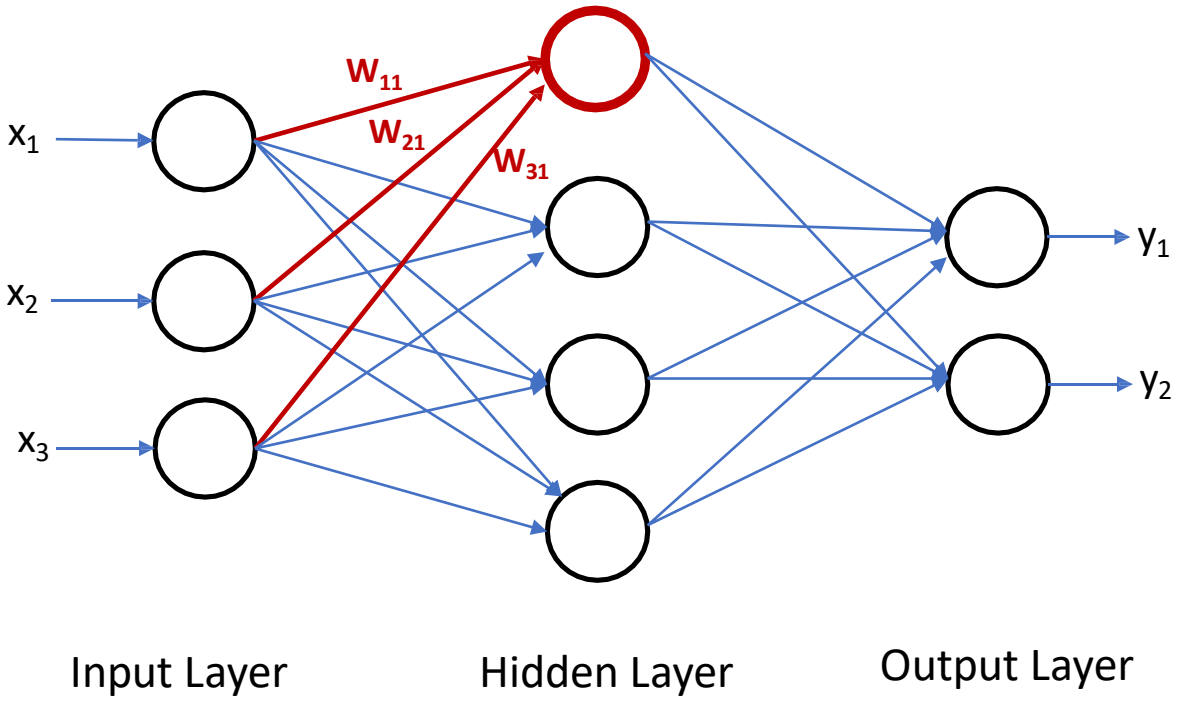


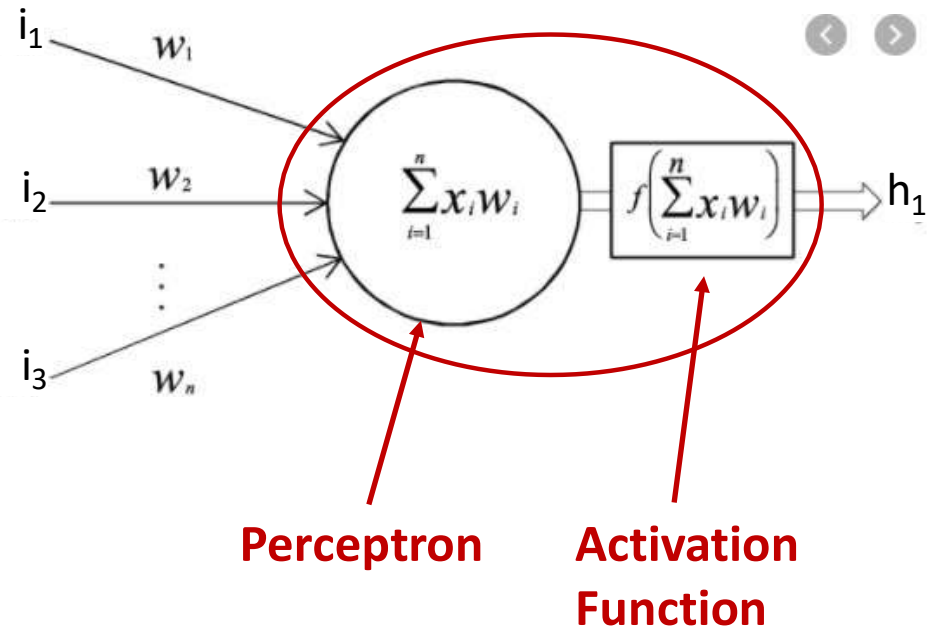
Summing Function (Σ)

- This computes the **induced field (net input)**:

$$z = \sum_{i=1}^{m} (x_i \cdot w_i) + b$$

- It adds all the weighted inputs and the bias.


x_1
x_2
x_3
W_{11}
W_{21}
W_{31}
y_1
y_2
Input Layer
Hidden Layer
Output Layer


i_1
i_2
i_3
w_1
w_2
w_n
$\sum_{i=1}^{n} x_i w_i$
$f\left(\sum_{i=1}^{n} x_i w_i\right)$
h_1
Perceptron
Activation Function

Activation Function φ(z)

- Applies a **non-linear function** to the result of the summation.

- Common activation functions:

  - ✓ **Sigmoid**: It **squashes** any real-valued input into a value **between 0 and 1**.

$$\frac{1}{1+e^{-z}}$$

  - ✓ **ReLU (**The Rectified Linear Unit): max(0,z)

Where, z is the input (e.g., weighted sum of inputs plus bias)

Activation Function φ(z)

- ✓ Softmax Function:

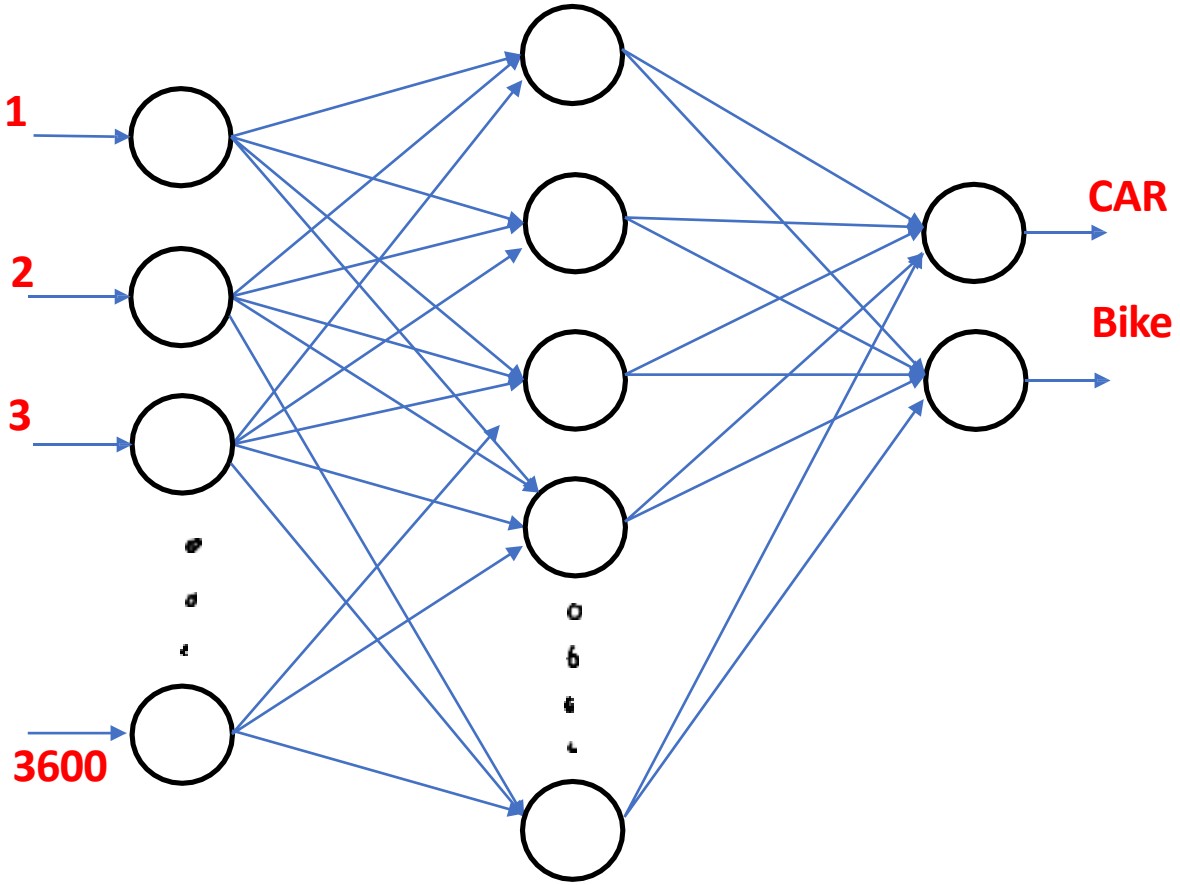
For a vector $\mathbf{z}$ of K real numbers, the softmax function $\sigma(\mathbf{z})$ is defined as:

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

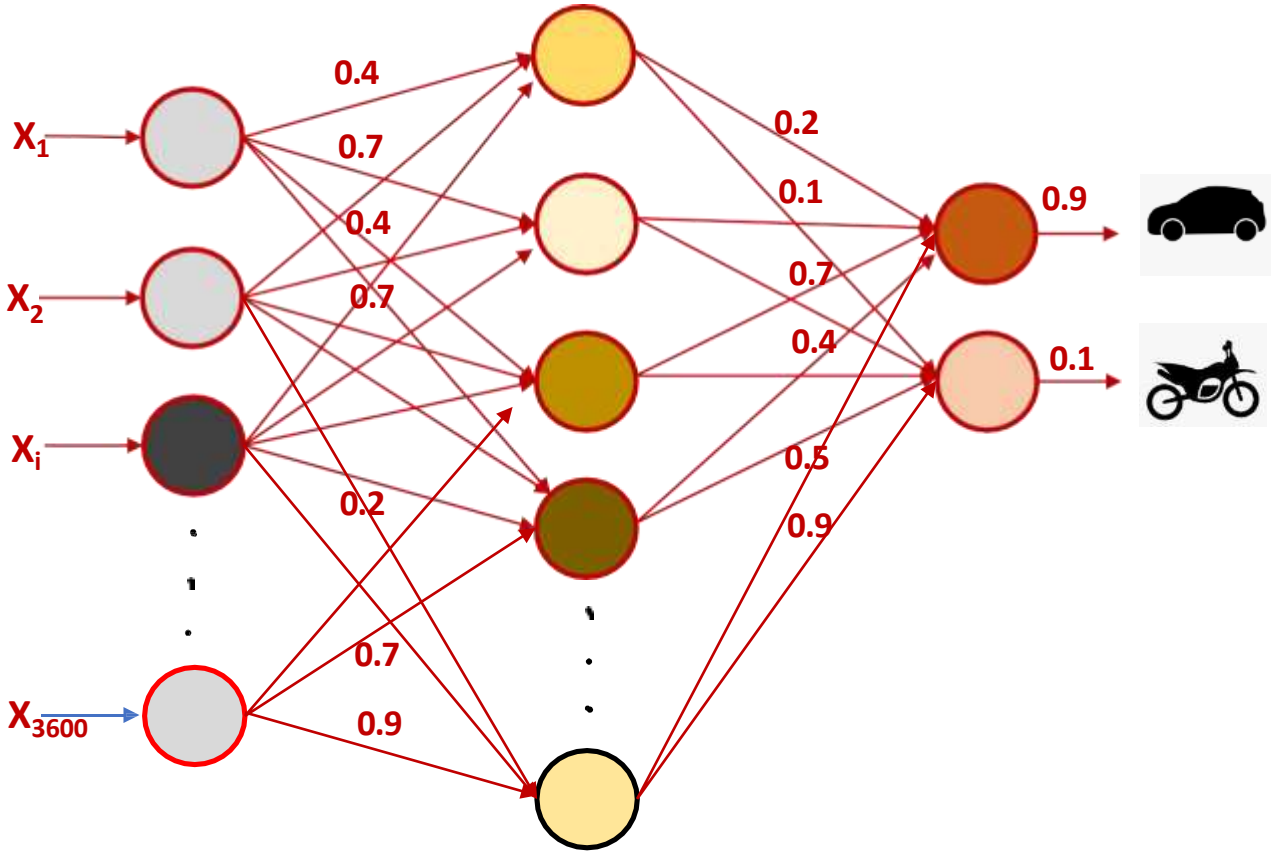
A Toy Example

60x60=3600

60x60=3600

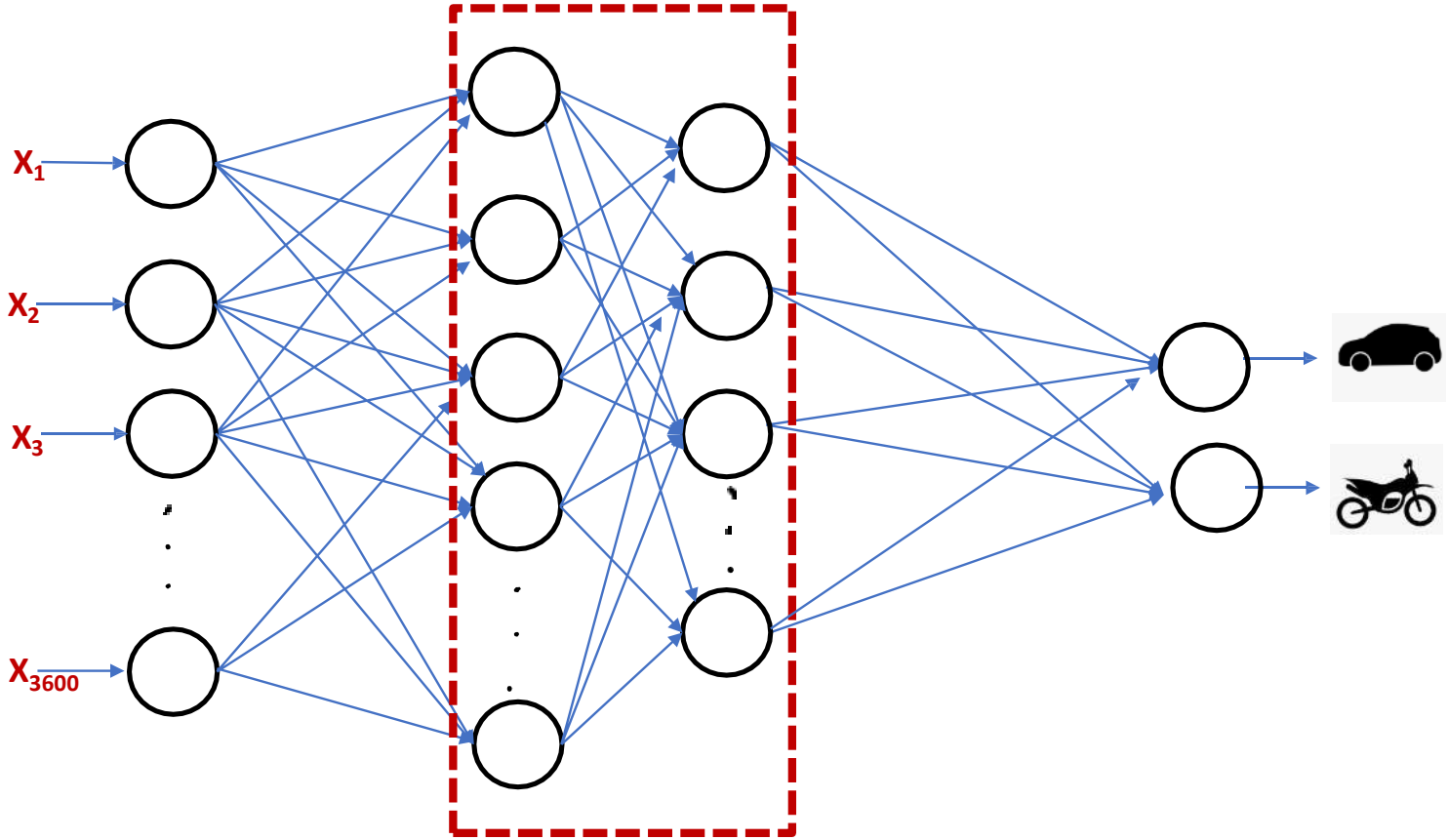


60x60=3600



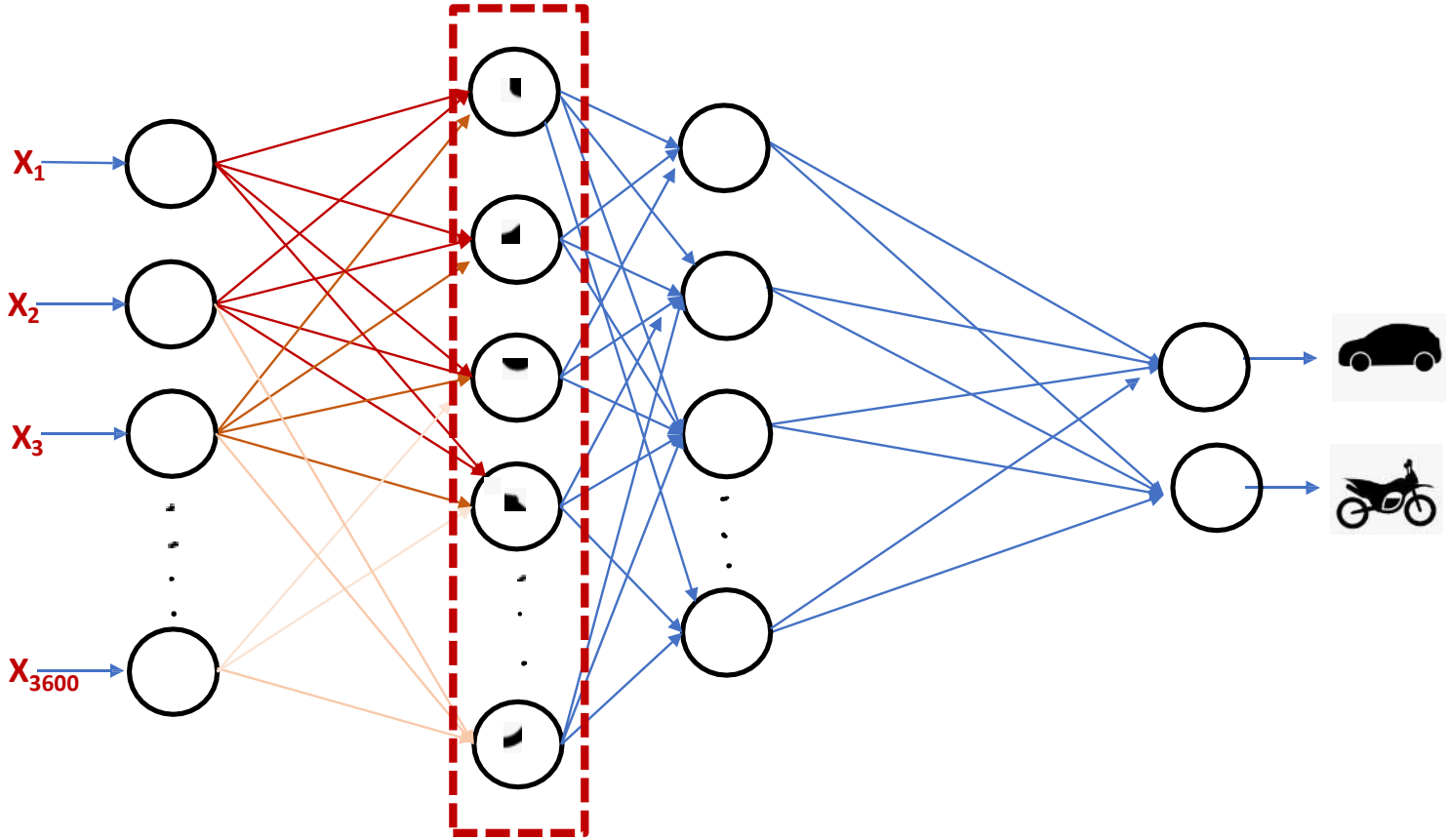
Forward pass

60x60=3600

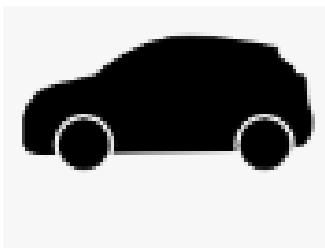


Multiple Hidden Layers

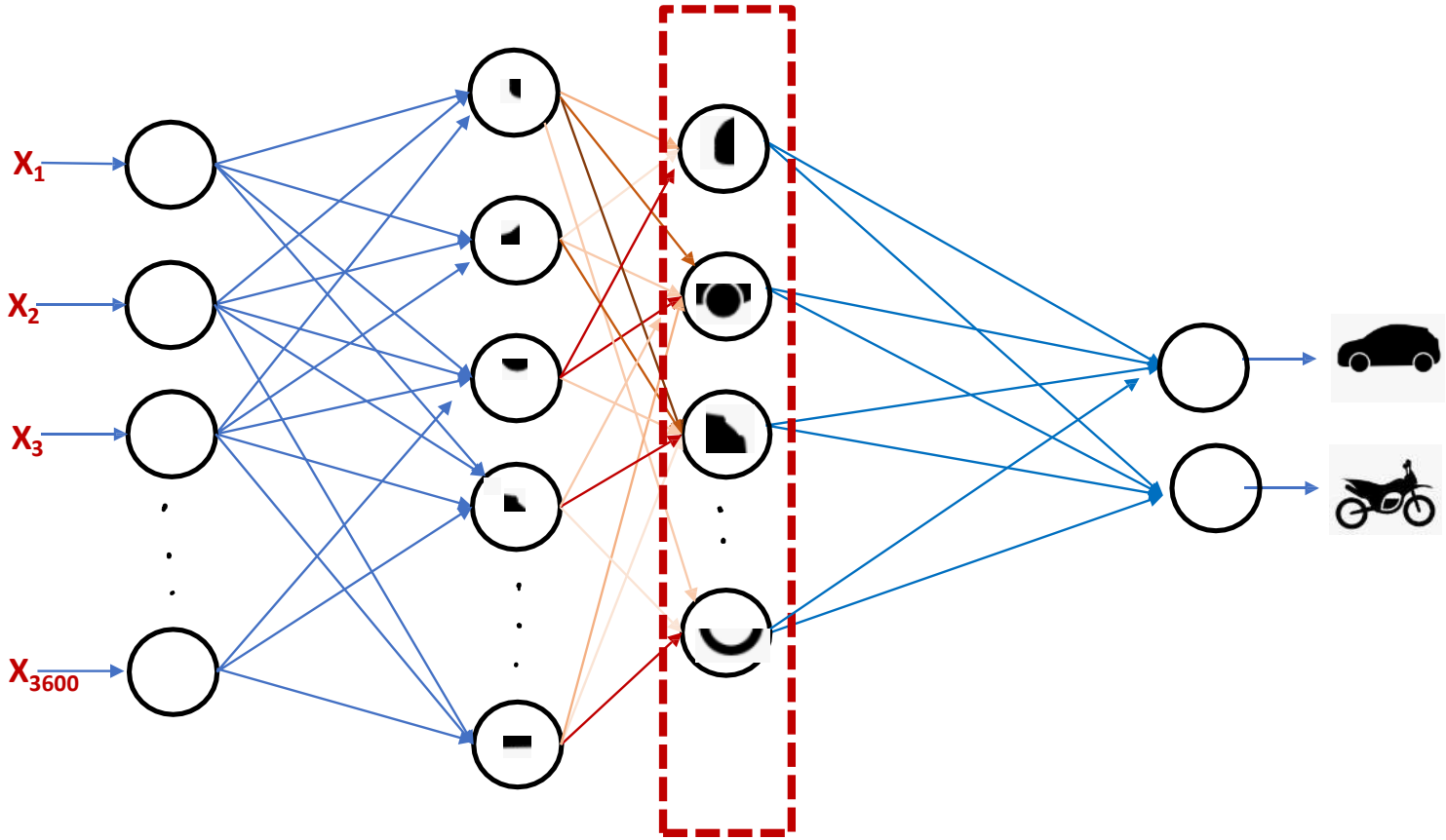
60x60=3600



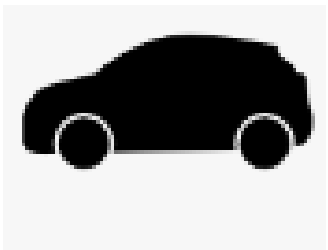
Lower level features



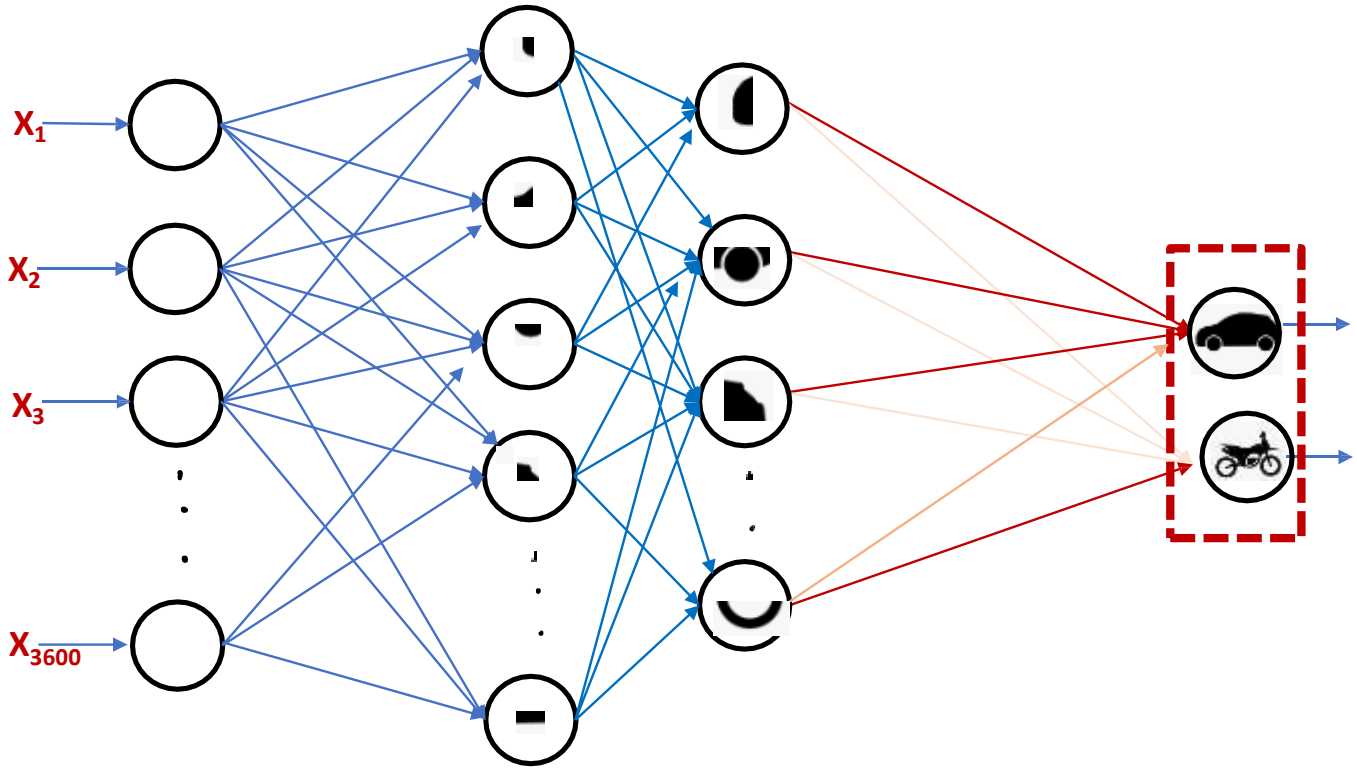
60x60=3600



Higher level features

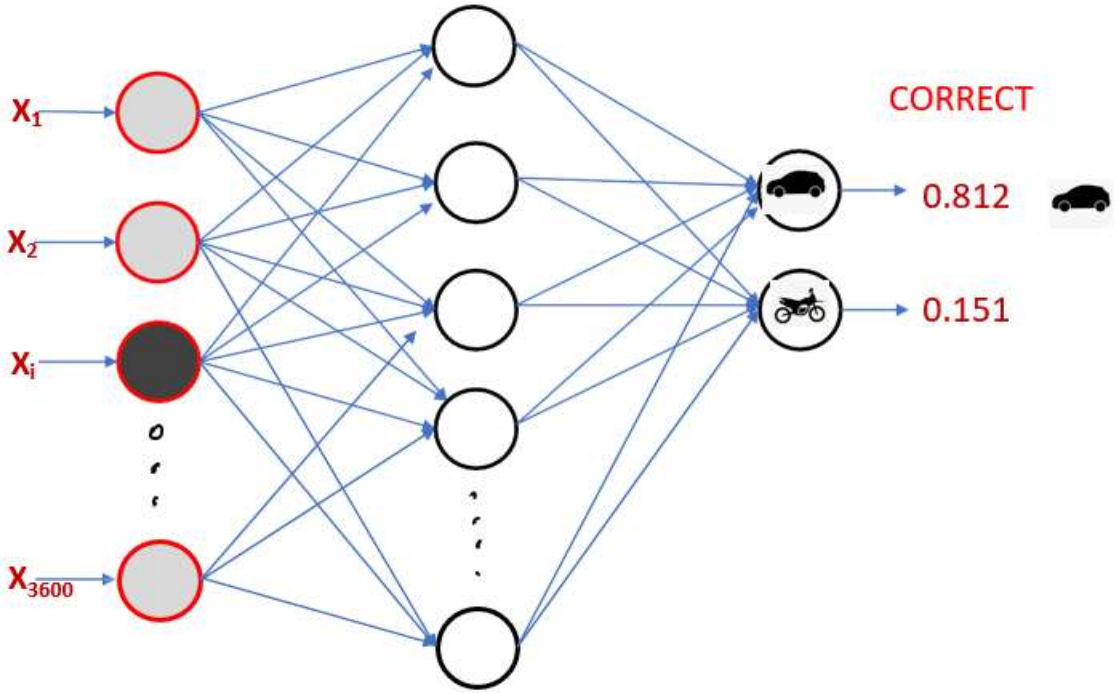


60x60=3600

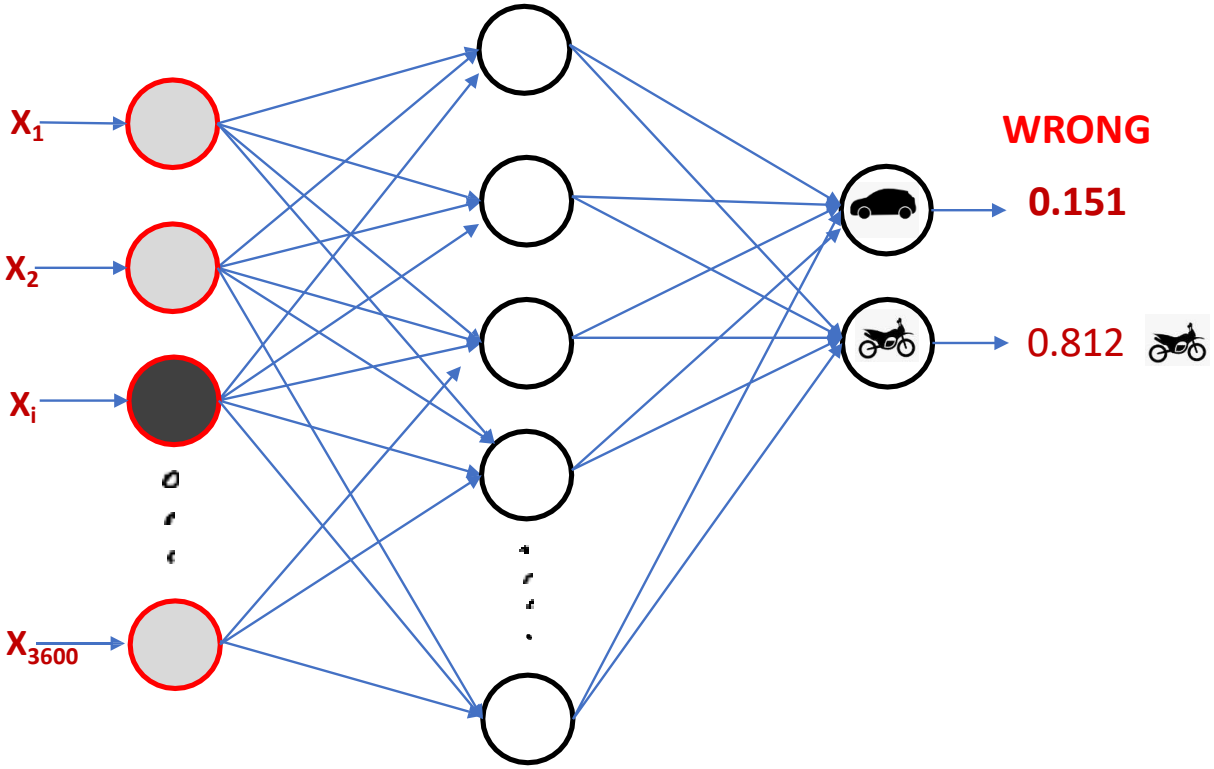


Higher level features

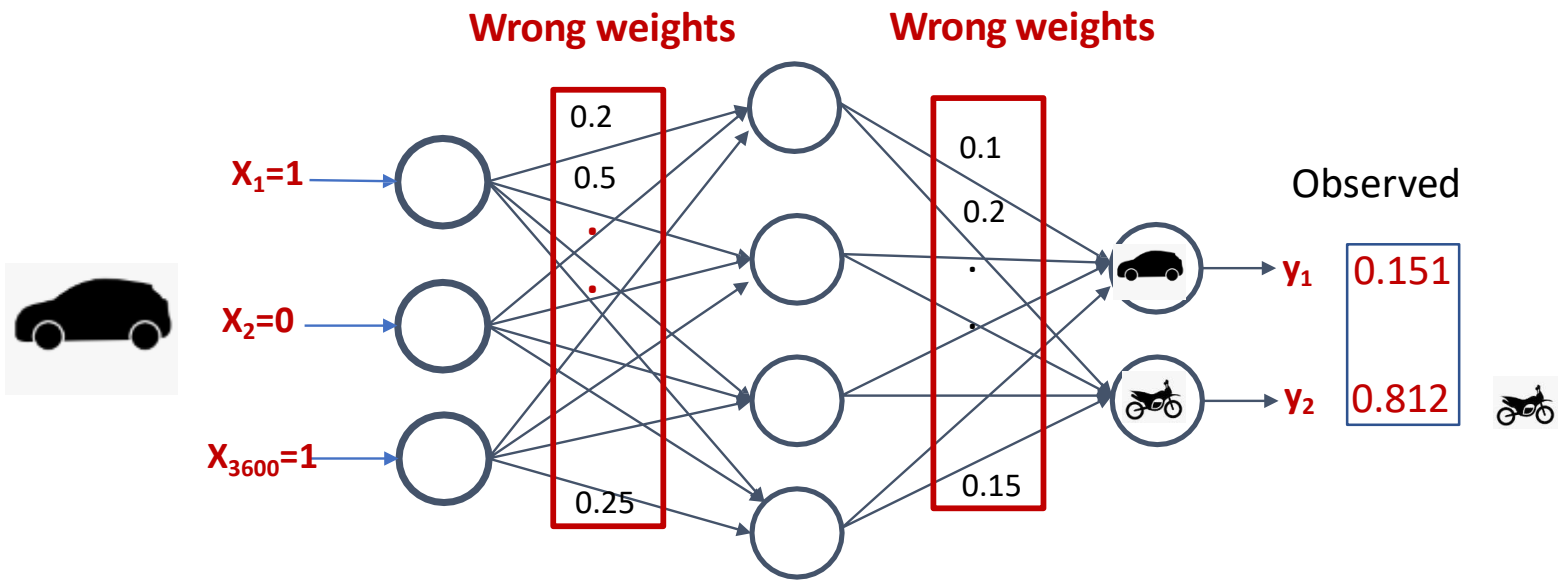
60x60=3600



60x60=3600



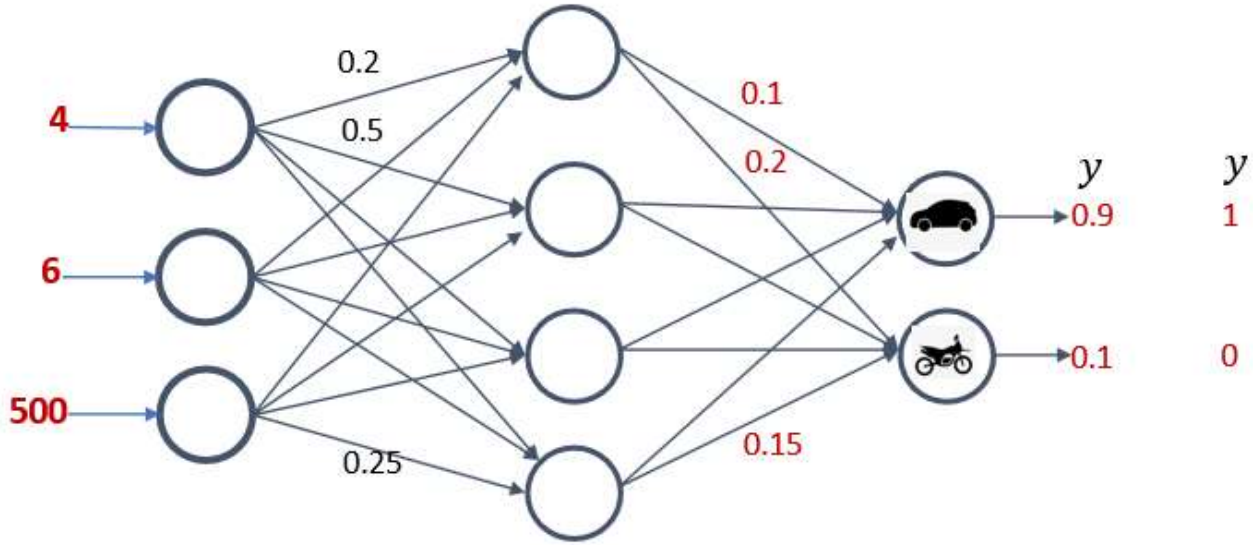
Why there is an error in prediction?



Two Commonly used Loss Functions are

- Mean Square Error – Standard Loss Function for Regression
- Cross Entropy Loss - Standard Loss Function for Classification

#Wheel	Height	Weight	
4	6	500	
4	5.5	600	
4	5	550	
2	3	200	
2	3.5	150	
2	4	250	



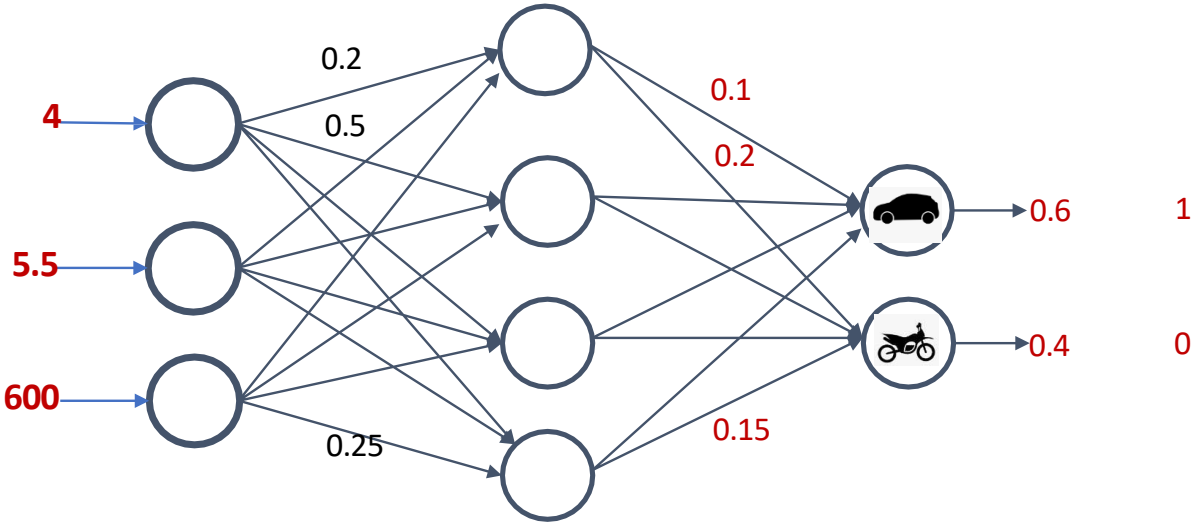
Error $E = (y - y)^2$

Error $E_{y_1} = (0.9 - 1)^2 = 0.01$

Error $E_{y_2} = (0.1 - 0)^2 = 0.01$

If these errors are not acceptable, then Backpropagate.

#Wheel	Height	Weight	
4	6	500	
4	5.5	600	
4	5	550	
2	3	200	
2	3.5	150	
2	4	250	



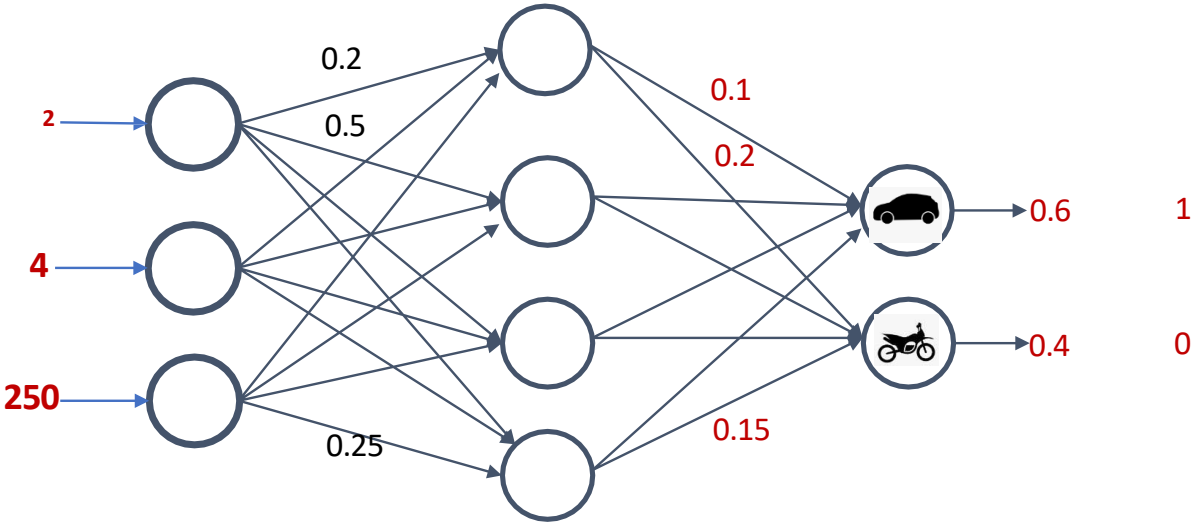
Error $E = (y - y)^2$

Error $E_{y_1} = (0.6 - 1)^2 = 0.16$

Error $E_{y_2} = (0.4 - 0)^2 = 0.36$

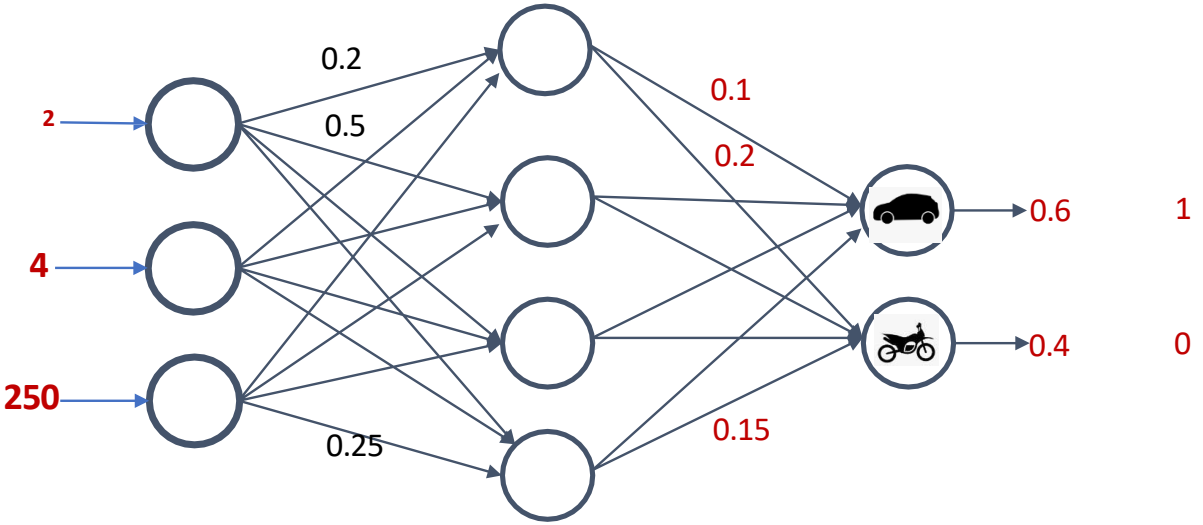
If these errors are not acceptable, then Backpropagate.

#Wheel	Height	Weight	
4	6	500	
4	5.5	600	
4	5	550	
2	3	200	
2	3.5	150	
2	4	250	



One complete cycle of training is called Epoch

#Wheel	Height	Weight	
4	6	500	
4	5.5	600	
4	5	550	
2	3	200	
2	3.5	150	
2	4	250	



Backpropagation after every sample is expensive.

Do it in batches.

Two Commonly used Loss Functions are

- Mean Square Error – Standard Loss Function for Regression
- Cross Entropy Loss - Standard Loss Function for Classification

PLATFORM

GPU powered cloud server -Google Colab

Jupyter Notebook -Google Colab

CNN Basics

CNN Basics – Kernels, Convolution, Pooling, Dropout

What is CNN?

CNN (Convolutional Neural Network) is a deep learning model mainly used for:

- Image classification
- Face recognition
- Object detection
- Medical image analysis
- Video processing

CNN is powerful because it can automatically detect:

- edges
- shapes
- textures
- objects

Basic Difference

ANN

Uses fully connected layers where every neuron connects to next layer neuron.

Best for:

- Numerical data
- Tabular data
- Prediction problems

CNN

Uses convolution layers with filters/kernels.

Best for:

- Images
- Videos
- Pattern recognition

Structure Difference

ANN Structure

Python

Run

```
Input → Hidden Layer → Hidden Layer → Output
```

All neurons connected.

CNN Structure

Python

Run

```
Input Image
→ Convolution
→ Pooling
→ Flatten
→ Dense
→ Output
```

1. Kernel / Filter

What is Kernel?

A **kernel** (also called **filter**) is a small matrix that scans over image pixels.

Example:

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

This kernel may detect vertical edges.

Common kernel sizes:

- 3×3
- 5×5
- 7×7

Why Kernels?

Different kernels learn different patterns:

- Edge detector
- Blur detector
- Corner detector
- Texture detector

In deep learning, kernels are **learned automatically** during training.

2. Convolution

What is Convolution?

Convolution means sliding the kernel over image and computing dot product.

At every location:

1. Multiply matching values
2. Add results
3. Store output pixel

This creates a **feature map**.

Example

Input image (3×3):

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel (2×2):

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Top-left convolution result:

$$(1 \times 1) + (2 \times 0) + (4 \times 0) + (5 \times 1) = 6$$

This continues across the image.

3. Activation Function (ReLU)

After convolution:

```python
ReLU(x) = max(0, x)
```

$$ReLU(x) = \max(0, x)$$

Negative values become 0.

Benefits:

- Non-linearity
- Faster training
- Reduces vanishing gradient

4. Pooling Layer

Pooling reduces image size while keeping important features.

It decreases:

- computation
- memory
- overfitting

Types of Pooling

A. Max Pooling

Take maximum value from region.

Example 2×2 block:

$$\begin{bmatrix} 1 & 5 \\ 2 & 3 \end{bmatrix} \to 5$$

Most common.

B. Average Pooling

Take average.

$$\frac{1+5+2+3}{4} = 2.75$$

5. Flatten Layer

After convolution + pooling, output is 3D.

Flatten converts to 1D vector.

6. Dense Layer

Normal fully connected neural network layer.

Used for final classification.

7. Dropout

What is Dropout?

During training, randomly turn off some neurons.

What Does `0.5` Mean?

```python
Dropout(0.5)
```

Means:

- During training, **50% of neurons** in that layer are randomly turned OFF.
- Remaining 50% stay active.

So in each training step, the network uses a different random set of neurons.

Why Use Dropout?

Without dropout:

- Model may memorize training data
- Performs well on training data
- Performs poorly on new test data

With dropout:

- Model learns general patterns
- Better performance on unseen data